An IoT-Enabled Multisensor Air Quality Monitoring Prototype with Edge-Based Composite Air Quality Score Computation and On-Demand AI-Assisted Mobile Interpretation

ABSTRACT - ***Low-cost air quality monitors on microcontrollers often just show the sensor numbers and stop there. The user has to figure out what it means. This prototype puts together an ESP32 board with a few sensors like the DHT11 for temperature and humidity, an MQ 135 gas sensor, a dust sensor, and a GPS module. There is also a mobile app made with Flutter.***

The board works out a score called CAQS by combining the dust reading, the gas sensor value, and temperature with some weights the team picked. It turns the score into one of six labels and sends everything as simple JSON over Bluetooth. No internet or cloud stuff is involved. The score is not an official air quality index because the inputs are not calibrated and the weights are just internal choices.

The app can also send the readings to Google Gemini when asked and show a plain language note about them. That part is only for help with understanding and not a real prediction tool. Tests in normal conditions and with extra particles showed the data format worked fine over both serial and Bluetooth. The category moved from good at a low score to hazardous at a high one.

The dust number came out way too high for indoor air though and it pretty much decided the whole score. It is near the top of what the sensor can report. No checks against reference tools or measurements of power use or range were done so this just shows the setup can run and not that the numbers are accurate. It feels like the dust part might need more work but that was not tested here.

KEYWORDS – ESP32, Air Quality Monitoring, Composite Air Quality Score, Bluetooth Classic, Flutter, Generative AI, Environmental Monitoring, GPS

I. INTRODUCTION

Low cost IoT nodes have been reported for this kind of thing. It seems continuous local environmental monitoring is becoming an engineering concern these days. Air quality affects human health comfort and productivity. There is one with a Raspberry Pi that acts as a web node for checking air. I think that part helps, but I am not totally sure how the whole setup works in practice. [1], an ESP32 that reads temperature and humidity [2]. An ESP32 node that uses a DHT11, plus an MQ-7 and an MQ-135 [3]. And there is also an ESP32 weather station [4]. Air temperature, moisture level, dust particles, and gas amounts change over time and across different places. Because of that, readings from one main weather or air station far away might not match what is happening in a specific room, a bus or car, or a nearby street. [4]. This creates a need for inexpensive sensors that monitor the immediate surroundings.Existing technologies that measure local environment awareness are based on either manual gauging or single-purpose analog gauges. Both of these systems are unable to perform continuous multifactor measurements and offer little or no analysis of the results when an anomaly occurs. Using an electronic device that displays information on a minuscule screen is no better than the analog gauges because it does not explain what the deviations mean.Falling prices of system-on-chip microcontrollers with built-in wireless interfaces have made hobbyist-level monitoring nodes affordable. An ESP32 microcontroller plus a DHT22 temperature-humidity sensor was used to report temperature and humidity data to the Blynk platform. [2], a development board of ESP32 with DHT11, MQ-7, MQ-135, that has driven OLED and LED displays [3], and an ESP32-WROOM DEVKIT station integrated BME280 weather sensor, MQ-135, and ML8511 UV sensor [4]. Together with cloud services such as ThingSpeak, Blynk, or Google Sheets, such nodes allow for real-time monitoring at a low cost, as in the Raspberry Pi node, which used ThingSpeak via esp32 and sensors. [1].

However, the weakness inherent to these systems is that they stop at the data collection and presentation stage. They either show the raw sensor values or a threshold-based colorization. None of the systems reviewed in Section II offer any natural-language interpretation of the observed state that might assist the end-user in responding to the observed situation.Narrowing this functionality gap does not necessarily require a machine learning model that would run on the device. A separate application could encode the sensor state as a narrowly-framed natural language question and report it, on demand, to a cloud-based generative model that would provide an appropriate context-based reply without requiring a separate classifier trained on domain-specific data, so long the exchanged messages are framed in a way that prevents the model from making up information or attempting to diagnose the cause of the observations. Note that such an approach would not be guaranteed to provide accurate information; therefore, the exchange should be framed as an explanation of the sensor values rather than an assertion of objective truth.

In this paper we describe a prototype air quality monitoring system with three components: (i) an ESP32 sensing node with a DHT11, an MQ-135 analog gas sensor, a GP2Y1010 optical dust sensor, and a NEO-6M GPS module, which computes a prototype-specific Composite Air Quality Score (CAQS) locally, then sends the readings, score, category, and GPS fields as newline-delimited JSON over Bluetooth Classic SPP; (ii) a cross-platform Flutter/Dart application that parses the stream and displays it on a live

dashboard; and (iii) an on-demand function in which the application sends the current readings to the Google Gemini API and displays a natural-language explanation of the results. The contributions are (1) a documented ESP32-to-phone Bluetooth pipeline whose JSON payload was sanity-checked against a USB-serial log; (2) an explicit, reproducible description of the CAQS and sensor-processing equations and their limitations; (3) an application design that combines a live visualization, GPS status, and on-demand AI interpretation; and (4) a functional evaluation under a baseline and a particulate-increased condition, with an explicit discussion of what the data do and do not show. We make no claim of regulatory accuracy, calibration, or forecasting performance.

II. LITERATURE SURVEY

All environmental monitoring systems described in the scientific literature have similar concepts: the microcontroller or single board computer that is connected to a low-cost sensor suite, wireless communication to send data, and either cloud computing or widgets for storing and displaying data. The variations among these systems are due to the selection of the processing platform, sensor suite, communication method to leave the node, finally, whether any sort of data interpretation has been carried out before the data reaching the end user. Single-board computers offer more processing power than microcontrollers and have been used as sensor nodes for this reason, as described next.Balasubramaniyan and Manivannan [1] built an IoT node based on a Raspberry Pi processor using a GrovePi+ shield and equipped with sensors for evaluating humidity, dust, MQ-5, MQ-7, and MQ-135 gas concentrations. The data was uploaded to the cloud via WiFi and hosted on ThingSpeak where MATLAB analytic software was used to process the information and post alerts to Twitter. The processing power was offloaded to the cloud to reduce the load on the IoT node. However, the system only supported four nodes, was based on the HTTP protocol, and did not have a client application.Many systems based on the ESP32 are low-cost solutions that rely on the ESP32's Wi-Fi to connect to the cloud or a widget platform. Adeagbo [2] used an ESP32 and a DHT22 sensor connected to the Blynk platform to indicate the temperature and humidity levels in his local area using a mobile widget. The result indicated that while the temperature value reported was 6 o C, the actual value was 4 o C (20% error), whereas the humidity level had an error of 2%. Omkar et al. [3] used an ESP32 microcontroller and DHT11, MQ-7, and MQ-135 sensors in an air quality monitoring system that used an OLED display, LED indicators, and Blynk or ThingSpeak platform but without giving quantitative results on a comparative basis. On the other hand, Uddin and Praharaj [4] The existing environment monitoring solutions designed for personal or small business use employ a common set of features. The first solution combined a BME280, an MQ-135, an ML8511 UV sensor, and a rain sensor on an ESP32-WROOM DEVKIT that logged to Google Sheets, served an HTML dashboard, and mirrored data to Blynk; their e-mail alert feature was not fully validated. The three main limitations of such solutions are (i) the requirement of a Wi-Fi link to a cloud or widget platform, (ii) a user interface implemented as a third-party widget service, a web page, or an LED/OLED display, and (iii) the absence of a natural-language explanation of the combined sensor state beyond fixed threshold checks..

The most recent related work [5] used an ESP32-WROOM node with DHT22, BH1750, and DSM501A sensors and MQTT over Wi-Fi to ThingSpeak. It is reported there, besides the calibration procedure against reference instruments, statistical error metrics, and measurements of latency, packet delivery, and power. However, its comparison with commercial monitors was based on datasheet specifications and not on parallel measurements [5]. This work provides what a quantitative evaluation of such a node entails: a reference-instrument comparison, calibration, repeated measurements, and network and power characterization. The prototype under evaluation here has not been subjected to these tests, and its results cannot be compared to the numbers reported in [5].The reviews of the field indicate the same pattern. IoT-based air quality monitoring systems usually consist of multiple sensors, a microcontroller, and a communicating module, the data from which are displayed on a monitoring interface [6]. In a survey of low-cost IoT pollution monitors, ESP-series microcontrollers were used in approximately 40% of the systems, and IoT cloud and web services were the preferred interface in approximately 53% [7]. Another review of low-cost indoor systems identified Wi-Fi as the prevalent communication technology and pointed to sensor calibration and energy consumption as common challenges associated with accuracy and usability [8]. A direct Bluetooth-to-phone design thus departs from the norm of the Wi-Fi and cloud pattern, and calibration is a recognized challenge of low-cost systems that the current prototype has not yet tackled.

III. PROPOSED METHODOLOGY

The system is implemented as an end-to-end pipeline. Environmental sensors are read by an ESP32 acquisition node that computes a prototype-specific composite score locally, then serializes it as JSON over a direct Bluetooth Classic serial connection and streams it up to paired smartphones; a Flutter app parses the JSON into a live dashboard. At the user's request, the Flutter application issues the current readings to a cloud-hosted generative-AI model that replies with an explanatory text. The individual pipeline stage descriptions are provided in the subsequent subsections.

A. Overall System Architecture

The system features five stages that are illustrated in Fig. 1. The sensor node consists of an ESP32-WROOM-32 development board interconnected on a breadboard with the DHT11, MQ-135, GP2Y1010, and NEO-6M modules (Section III-B); the GPS module is part of the sensor node and its fields are sent in the same payload. The ESP32 reads each sensor and calculates the Composite Air Quality Score (CAQS) and its category locally (Section III-C) and serializes the result as JSON. A Bluetooth Classic Serial Port Profile (SPP) link carries the payload to a paired Android device that is devoid of Wi-Fi or a cloud broker. The Flutter application receives and parses the stream and displays the readings, the CAQS, and the GPS state. Only on the explicit request of the user, does the application send one HTTPS request to the Google Gemini API whose response is displayed as an environmental explanation (Section III-F). The Gemini model does not participate in the CAQS calculation which is only made on the ESP32.

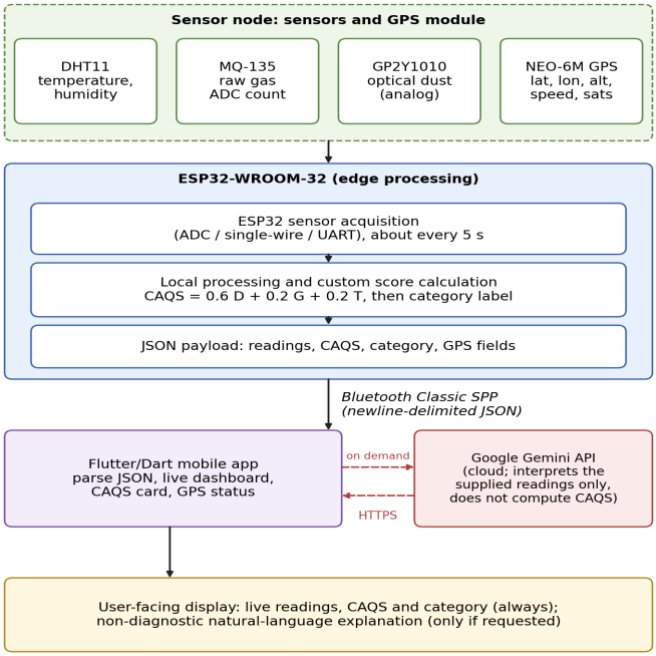


Fig. 1: Overall system architecture and data flow of the prototype

B. Environmental Sensing Unit

The sensing unit consists of four modules connected to the ESP32 (Table I). DHT11 is a low-cost digital sensor that provides temperature and relative humidity over a single-wire interface; its resolution and accuracy are limited. MQ-135 is an analog metal-oxide gas sensor module that responds to several gases, for example ammonia, nitrogen oxides, benzene, smoke, and carbon dioxide, but it can not distinguish between them [3]. Its analog output is read by the 12-bit ESP32 ADC as a raw count from 0 to 4095. This count should not be interpreted as a calibrated concentration; thus, the gas channel is used only as a relative indication and is never expressed in ppm. The GP2Y1010 is an infrared optical dust sensor, whose analog output is converted by the firmware to a voltage and then to a dust-density estimate in µg/m³ using Eqs. (1) and (2); that is, this estimate is not a calibrated PM2.5 or PM10 value (Section III-C). Low-cost light-scattering particulate sensors, including the GP2Y1010AU0F, are not inherently calibrated and must be evaluated against reference instruments before use [9],

The GP2Y1010AU0F works in a pulsed mode where its output voltage is to be sampled at a specific moment of each pulse [10]. The NEO-6M GPS module outputs latitude, longitude, altitude, ground speed, and satellite count over a UART, so that a reading can be geotagged if a satellite fix exists. The GPS module is part of the sensor node, and its fields are transmitted in the same JSON payload. Since these parts are low-cost, none of the gas, dust, temperature, or humidity channels is claimed to have laboratory-grade accuracy, and the score defined in Section III-C is not a certified regulatory index.

TABLE I. SENSOR AND MODULE INTERFACES OF THE PROTOTYPE

Module	Quantity	Interface and firmware use
DHT11	Temperature, relative humidity	Single-wire digital input; values used directly
MQ-135	Relative gas indication	Analog output to 12-bit ESP32 ADC; raw count 0–4095 used as G
GP2Y1010	Optical dust sensor	Analog output to ESP32 ADC; converted to V_D and dust estimate D (Eqs. 1–2)
NEO-6M	Latitude, longitude, altitude, speed, satellites	UART; fields sent in the JSON payload (zero until a fix)

C. ESP32-Based Data Acquisition and Composite Air Quality Score

The firmware reads the sensors approximately every five seconds: it samples the DHT11, reads the MQ-135 and GP2Y1010 analog channels through the ESP32 ADC, and reads the NEO-6M over UART. From these inputs it computes the dust estimate, the CAQS, and a category string (Eqs. (1)–(3)) and packs every reading set into a single-line JSON object with the fields temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, and satellites. The key names aqi and airQuality are retained from the firmware and application code, but they carry the CAQS and its category label. The object is written to the USB serial port at 115200 baud and to the Bluetooth Classic SPP serial interface; the two outputs carried the same payload during testing (Section IV). The payload contains no timestamp field. The GPS fields remain zero until a satellite fix is obtained, which did not happen during the indoor tests (Section IV).

Dust estimate. The GP2Y1010 analog output is converted to a voltage by Eq. (1), where ADCD is the 12-bit ADC count (0 to 4095) and 3.3 V is taken as the ADC full-scale voltage. The dust-density estimate D in µg/m³ is then calculated by Eq. (2):

$$VD = ADCD \times (3.3 / 4095) \quad \text{----->} \quad (1)$$

$$D = \max(0, (VD - 0.1) \times 1000) \quad \text{----->} \quad (2)$$

where VD is in volts, 0.1 V is a fixed offset, and 1000 µg/m³ per volt is a fixed slope in the firmware. These constants constitute an uncalibrated linearization; they were not obtained by comparison with a reference particle instrument, and the outcome is therefore not a measured concentration of PM2.5 or PM10. Since the ADC range is 0 to 3.3 V, D cannot exceed $(3.3 - 0.1) \times 1000 = 3200$ µg/m³, and ADC nonlinearity at full scale may additionally distort the high values.Gas inputs. The gas value G in Eq. (3) is simply the raw MQ-135 ADC count (0 to 4095); it was not converted to sensor resistor, concentration, or ppm, but was used directly. The count depends on sensor warm-up time, temperature, and humidity, as well as on the gas mixture. A calibration against a reference gas was not done in this work.Composite Air Quality Score. The prototype computes the dimensionless score S of Eq. (3), where T is the DHT11 temperature in °C:

$$S = 0.6\,D + 0.2\,G + 0.2\,T \quad \text{---> (3)}$$

The weights 0.6, 0.2 and 0.2 are internal design choices of the prototype; they were not determined from a validated health- or regulatory-based methodology. The three inputs have different units and ranges (µg/m³-based estimate, unitless ADC count, and °C) and are not normalized. The score is reported as an integer (the reported values are consistent with truncation) and mapped to a category label, evaluated in this order: GOOD ($S \leq 50$), MODERATE ($S \leq$

100), UNHEALTHY FOR SENSITIVE GROUPS ($S \leq 150$), UNHEALTHY ($S \leq 200$), VERY UNHEALTHY ($S \leq 300$), and HAZARDOUS ($S > 300$). These labels and thresholds are reminiscent of the category names of regulatory air quality indices, but S is not a regulatory AQI: it is not calculated from pollutant concentrations over averaging periods using official breakpoints, and a CAQS category does not imply the corresponding regulatory category or health effect. For this reason the output is called the CAQS in this paper, although the JSON field name is aqi. Because T is added directly, a warm environment raises S even if the air is clean, and the large ranges of D and G compared with T mean that dust and gas dominate S in practice.Gas and optical dust sensors are sensitive towards temperature, humidity, airflow, and aging, and they require warm-up and calibration in relation to a reference instrument in order for their readings to be interpreted quantitatively. A such comparison was not performed in this work (Section V).

D. Device-to-Application Communication (Bluetooth)

Rather than a Wi-Fi uplink, the ESP32 is acting as a Bluetooth Classic SPP device announced as "ESP32_AQI." On the application side, the flutter_bluetooth_serial_plus package is employed: it initially checks Bluetooth is enabled, then scans the list of bonded devices for "ESP32_AQI" and opens connection using its address. It follows that the module must be paired in advance from the phone's Bluetooth settings. Once connected, the app decodes each incoming chunk as UTF-8 and appending to an internal buffer, and any time the buffer has a newline character, it extracts the text up to the newline, trims it, and passes it to a JSON decoder, and the remainder keeps in the buffer. This buffering is needed because SPP delivers an arbitrary byte stream, not message-framed packets, so one JSON object may arrive split across several callbacks. The only internet access in the system is initiated by the phone application, on demand, when the user requests the AI analysis (Section III-F). In this implementation the ESP32 does not use Wi-Fi and has no direct internet or cloud connection.

E. Flutter Application

The Dashboard is the single-page Flutter application written in Dart with the same code base potentially targeting Android, Windows, and Linux, although the results in Section IV were obtained on Android. It deliberately keeps the architecture simple and focuses on a single stateful widget: each incoming Bluetooth message is parsed into fields of typed data (temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, satellites) stored within the widget state, and each successful parsing refreshes the interface by Flutter's setState() mechanism. The interface includes a Bluetooth connection card showing the device name and connection status, a large score card indicating the CAQS and its category with corresponding colors (green, orange, deep orange, red, purple, and brown) across six severity bands, separate cards for temperature, humidity, gas, and dust, a GPS information card, and an AI-analysis button with a result panel. In the screenshot of Fig. 2 the score card is labeled "Air Quality Index" indicating the CAQS defined in Section III-C. The evaluated version does not store the past readings over sessions and does not plot the time-series charts, so each screen only reflects the most recent parsed reading. A later native Android re-implementation (using Kotlin with Jetpack Compose) includes the local history storage and trend charts, but was not part of the tests reported here.

TABLE II SOFTWARE PACKAGES USED IN THE FLUTTER APPLICATION

Component	Package	Purpose
Bluetooth link	flutter_bluetooth_serial_plus 0.5.x	Discovers the paired ESP32_AQI device, opens a Bluetooth Classic SPP connection, and streams the JSON data.
AI HTTP client	http 1.6.x	Sends the constrained prompt to the Gemini generateContent REST endpoint and parses the response.

F. Environmental Analysis Using AI (Gemini)

The application decouples deterministic and generative functions. The deterministic function is the CAQS and category computed by the ESP32 firmware (Section III-C); the application only displays it, and it is available at any time when the Bluetooth link is up, without internet access or an AI service. The generative function is on-demand, AI-assisted interpretation: when the user presses the "AI Analyze" button, which is only available while connected, the application constructs a text prompt containing the current temperature, humidity, gas ADC value, dust estimate, CAQS and category, and GPS status, and sends it over HTTPS to the Google Gemini generateContent endpoint. The prompt requires the response to use fixed headings (Overall Assessment; Risk Level, which is LOW, MODERATE, HIGH, or CRITICAL; Key Observations; Possible Causes; Precautions; Health Considerations; Environmental Recommendations; Short-Term Prediction; and Important Warning) and instructs the model to use only the values supplied, to treat the CAQS as a custom score and not a regulatory index, to treat the gas value as an ADC reading and not a concentration, and not to provide a medical diagnosis. The response text is displayed verbatim in a result card. The Gemini model does not calculate the sensor score; this is done only on the ESP32. The "Short-Term Prediction" section is text generated by the language model from a single set of readings; it is not produced by a validated time-series forecasting model and must not be interpreted as a pollution forecast. The risk level and the health-related statements in the response are likewise text generated and not validated assessments, and there is no claim that they are accurate or consistent.

G. Data Processing and Decision Flow

The end-to-end process takes the following form: (1) the ESP32 microcontroller reads the DHT11, gas, dust, and GPS sensors; (2) the firmware calculates the dust estimate, CAQS, and category string (Eq. (1)-(3)); (3) the measurements are formatted into a JSON single-line object; (4) the object is pushed through the USB serial interface and Bluetooth Classic SPP; (5) the input-stream listener in the Flutter application gathers the incoming bytes until a new line is detected; (6) the message is parsed as JSON, and the sensor, CAQS, and GPS fields are updated; (7) the interface is rebuilt, and the cards are re-painted; (8) when the user presses "AI Analyze" while being connected, the current

prompt is formed as per the description of the constrained prompt of Section III-F; (9) the prompt is sent to the Gemini API over HTTPS, and the text response is extracted; and (10) the response text is placed in the result card. The steps (1)-(7) are repeated as long as Bluetooth messages arrive at the approximate rate of once every five seconds, whereas steps (8)-(10) are taken upon user request.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

The prototype was only tested at a functional level because there was no certified reference instrument available for calibration/comparison. The results below are therefore direct observations of the ESP32 output, the Flutter application, and the Gemini response; they demonstrate feasibility but do not establish measurement accuracy. Table III lists readings from two test conditions recorded with the same firmware version. The first is an example of a low-pollutant indoor baseline, read from the Arduino IDE serial monitor and compared against the Bluetooth JSON payload; the gas ADC reading was 0, the dust estimate was 0, and the firmware reported a CAQS of 5 (GOOD). The second was recorded in the Flutter application after exposure to a higher-particulate environment; the gas ADC reading was 186, the dust estimate was about 2947–2967 µg/m³, and the reported CAQS was 1823 (HAZARDOUS). Neither test was performed in a controlled manner. Only representative values are reported for each condition and the number of samples is not stated, so means and standard deviations are not specified and the values cannot be used for calibration. The reported scores correspond with Eq. (3): for the baseline, $0.2 \times 28.9 = 5.78$, reported as 5; for the elevated condition, the application screenshot (Fig. 2) shows a dust estimate of 2967.91 µg/m³, and $0.6 \times 2967.91 + 0.2 \times 186 + 0.2 \times 29.3 = 1823.8$, reported as 1823. The category therefore changed in the expected direction with changes in the inputs, and the new category was displayed correctly in the application.

TABLE III. REPRESENTATIVE SENSOR READINGS UNDER TWO TEST CONDITIONS

Parameter	Baseline (Indoor, Serial Log)	Elevated Condition (App Test)
Temperature (°C)	28.90	29.3
Humidity (%)	70.00	70.0
Gas (raw ADC)	0	186
Dust estimate (µg/m³)	0.00	~2947–2967
CAQS (on-device score)	5	1823
CAQS category	GOOD	HAZARDOUS
GPS satellites / fix	0 / no fix	0 / no fix

Plausibility of the dust estimate. A dust estimate of about 2950 µg/m³ is orders of magnitude above values typical of indoor air and is not plausible as a PM2.5 or PM10 concentration. Under the Eqs. (1) and (2) the largest possible estimate is 3200 µg/m³, reached at the ADC full-scale voltage, and a value of 2967.91 µg/m³ corresponds to VD ≈ 3.07 V, about 93% of full scale. The reading is thus close to the upper bound of the conversion range and is compatible with a saturated or poorly scaled analog signal (for example from the sensor output range, ADC attenuation or nonlinearity, or the sensor sampling), and not with a physical measured concentration. The cause was not investigated in this work. Because the dust term has weight 0.6, it contributed $0.6 \times 2967.91 \approx 1780.7$ of the total of about 1823.8 (about 97.6%), while the gas term contributed 37.2 and the temperature term about 5.9. The elevated-condition CAQS is thus almost entirely determined by the dust estimate, and its numerical value should not be interpreted as a severity level. A gas reading of 0 in the baseline condition is also unusual for a powered MQ-135 module and may indicate insufficient warm-up or an input-range problem; this was likewise not investigated.During the test sessions, newline-based framing in the application's stream buffer recovered complete JSON objects, and the payload received over Bluetooth matched the payload shown on the USB serial monitor at the same time. This is a functional check of message framing only: packet delivery ratio, latency, throughput, and communication range were not measured. The GPS module obtained no satellite fix in either (indoor) test session; the application stayed in the "Waiting for satellites…" state, and the latitude, longitude, altitude, and speed fields were parsed as zero. The absence of a fix is attributed to the indoor test environment, but this attribution was not verified outdoors, and GPS accuracy was not evaluated.



Fig. 2: Application screenshot during the elevated-particulate test (CAQS 1823; dust estimate 2967.91 µg/m³). The on-screen label "Air Quality Index" denotes the CAQS.

The on-demand AI analysis was exercised with the Gemini API while the application was connected to the sensor node. Because the output is free text and not a numeric score, no accuracy percentage can be given. In the sessions performed, the responses followed the required headings, used one of the four risk levels, and described the score as a custom score and the gas value as an ADC reading, as the prompt instructs. This is a qualitative observation of format

compliance; the number of responses examined is not reported here, and the observation is not an evaluation of correctness, consistency across repeated requests with identical inputs, or forecasting ability.

V. LIMITATIONS AND FUTURE EXPERIMENTS

Measurement validity. The gas input is a raw ADC count, not a concentration; the dust value is an uncalibrated voltage-based estimate, and neither has been compared to a reference instrument. The sensors are sensitive to warm-up, temperature, humidity, airflow, and age, and the DHT11 is a low-cost sensor with limited resolution and accuracy. Correct operation of the optical dust sensor also depends on its LED pulse timing and sampling which were not independently verified [10], and the elevated dust estimate is near the upper limit of its conversion range and physically unreasonable.Score validity. The CAQS weights are internal design options without a validated derivation; the inputs are not normalized; temperature is included as an additive term; and category thresholds are applied to a dimensionless score. The CAQS is therefore not equivalent to a regulatory AQI and should not be used for health or compliance purposes.Evaluation scope. Only two uncontrolled conditions were recorded; no repeated measures, reference monitor, or statistical analysis was made. Latency, packet delivery, Bluetooth range, power consumption, battery life, GPS accuracy, and AI-response correctness were not measured. The Bluetooth architecture requires pairing and short range, and provides no remote access; the tested application version stores no history; the AI function needs an internet connection on the phone, and a third-party service; and testing was only on Android. Planned experiments. The following experiments are planned and have not been carried out: (1) repeated measurements under controlled conditions, (2) baseline, moderate-change and elevated-particulate conditions, (3) outdoor GPS validation with a valid fix, (4) Bluetooth tests at a number of measured distances, (5) packet-loss and latency evaluation, (6) power and current measurements, (7) comparison with a suitable reference monitor from which MAE and RMSE can then be computed, and (8) tests of repeatability and of temperature and humidity influence.

VI. CONCLUSION AND FUTURE WORK

This paper introduced a prototype Bluetooth-based ESP32 air quality monitor, which combined a DHT11, an MQ-135 gas sensor, a GP2Y1010 dust sensor, a NEO-6M GPS module, and a Flutter application with optional on-demand AI-assisted interpretation via the Gemini API. The functioning was demonstrated: the JSON payload was transferred over USB serial and Bluetooth Classic SPP, the application's newline-based buffering parsed the messages, and the CAQS and its category changed in the expected direction from a baseline (5, GOOD) to a particulate-increased condition (1823, HAZARDOUS). The Gemini function output structured, non-diagnostic text in the tested sessions; its correctness and consistency were not evaluated. These results demonstrate the functional viability of a direct device-to-phone architecture and not measurement accuracy: the gas and dust channels are uncalibrated, the elevated dust estimate is implausible and near the conversion limit, and the CAQS is a prototype-specific score dominated by the dust term. Future work will calibrate the sensors against a reference monitor, redesign the score from normalized, pollutant-based inputs, carry out the experiments listed in Section V, extend the application with history storage and trend charts (a native Android re-implementation with these functions has been developed but was not part of the tests reported here), and evaluate the AI responses formally.

REFERENCES

[1] C. Balasubramaniyan and D. Manivannan, "IoT Enabled Air Quality Monitoring System (AQMS) using Raspberry Pi," Indian J. Sci. Technol ., vol. 9, no. 39, pp. 1–5, Oct. 2016. DOI: 10.17485/ijst/2016/v9i39/90414.

[2] A. A. Adeagbo, "IoT based environment monitoring system using ESP32," arXiv:2405.14047, May 2024. DOI: 10.48550/arXiv.2405.14047.

[3] N. Omkar, S. Rahul, R. Vijaykrishna, N. Swapna, and M. R. Sura, "Smart environmental monitoring using ESP32 microcontroller," IOSR J. Electron. Commun. Eng., vol. 19, no. 3, ser. 1, pp. 07–12, May-Jun. 2024, doi: 10.9790/2834-1903010712.

[4] M. S. Uddin and M. K. Praharaj, "IoT based weather monitoring system using ESP32 microcontroller," Int. J. Adv. Res. Sci. Commun. Technol., vol. 5, no. 2, pp. 693–701, Oct. 2025, doi: 10.48175/IJARSCT-29283.

[5] M. S. Bakare and K. Abubaker, "IoT-based indoor environmental monitoring system using multi-parameter sensing and ESP32-WROOM integration," Discover Electron., vol. 3, Art. no. 6, 2026, doi: 10.1007/s44291-026-00157-3.

[6] J. Saini, M. Dutta, and G. Marques, "Indoor air quality monitoring systems based on Internet of Things: A systematic review," Int. J. Environ. Res. Public Health, vol. 17, no. 14, Art. no. 4942, July 2020, doi: 10.3390/ijerph17144942.

[7] O. Alsamrai, M. D. Redel-Macías, S. Pinzi, and M. P. Dorado, "A systematic review for indoor and outdoor air pollution monitoring systems based on Internet of Things," Sustainability, vol. 16, no. 11, Art. no. 4353, 2024, doi: 10.3390/su16114353.

[8] J. Peixe and G. Marques, "Low-cost IoT-enabled indoor air quality monitoring systems: A systematic review," J. Ambient Intell. Smart Environ., vol. 16, no. 2, pp. 167–180, 2024, doi: 10.3233/AIS-220577.

[9] M. Ghamari, H. Kamangir, K. Arezoo, and K. Alipour, "Evaluation and calibration of low-cost off-the-shelf particulate matter sensors using machine learning techniques," IET Wirel. Sens. Syst., vol. 12, no. 5–6, pp. 134–148, 2022, doi: 10.1049/wss2.12043.

[10] K. Bučar, J. Malet, L. Stabile, J. Pražnikar, S. Seeger, and M. Žitnik, "Statistics of a Sharp GP2Y low-cost aerosol PM sensor output signals," Sensors, vol. 20, no. 23, Art. no. 6707, Nov. 2020, doi: 10.3390/s20236707.